

Jobra AI

Your gated, members-only job feed for **tech / software / cybersecurity** roles across the **U.S. and Canada**. Fresh listings pulled hourly, behind an email login, with payments designed to bolt on later.

Built to run at **\$0** until you start earning.

What's in here

```
jobra-ai/
├── index.html                ← the branded front-end (deploy this to Netlify)
├── supabase-setup.sql       ← run once in Supabase to create the database
├── ingest/
│   ├── ingest-adzuna.mjs    ← pulls fresh US+CA tech jobs into Supabase
│   └── package.json
├── supabase/functions/
│   └── stripe-webhook/index.ts ← auto-activates members when they pay (deploy to
└── .github/workflows/
    └── ingest.yml           ← runs the ingestion every hour, for free
```

How the pieces fit: **Netlify** serves the page → **Supabase** handles login + stores jobs and members → the **ingestion script** (run hourly by **GitHub Actions**) keeps the jobs fresh. A `status` flag on each member is the single switch that grants/revokes access, and that's where Stripe plugs in later.

Setup (about 30 minutes, all free)

1. Database (Supabase)

1. Create a free project at supabase.com.
2. Open **SQL Editor** → **New query**, paste all of `supabase-setup.sql`, and **Run**.
3. Edit the last line first to use *your* email so you can test, e.g. `('me@gmail.com', 'Founder', 'active')`.

4. Go to **Settings → API** and copy two values: the **Project URL** and the **anon public** key.
5. Under **Authentication → URL Configuration**, add your site URL to **Redirect URLs** (use `http://localhost` while testing, then your Netlify URL once deployed).

Email magic-link login is on by default, with no extra config needed.

6. **Log members out after 30 days** (their data is kept). Go to Authentication → Sessions and set “Time-box user sessions” to 720 hours (30 days). After that they sign in again with a fresh magic link; their résumé, saved jobs, and subscription status all stay in the database untouched. The app also enforces this on its side as a backup.

2. Front-end (Netlify)

1. Open `index.html` and paste your Project URL + anon key into the `JOBRA_CONFIG` block at the top.
2. Deploy: drag the `jobra-ai` folder onto **app.netlify.com/drop**, or connect the repo. Free.
3. Copy your live Netlify URL back into Supabase’s **Redirect URLs** (step 1.5).

That’s the gated app working. Sign in with the email you added as `active` and you’ll see the feed; any other email gets the “not active yet” screen.

Preview first: before adding keys, just open `index.html` in a browser, where it runs in **demo mode** with sample jobs so you can see the design.

2b. Custom domain (jobrajobs.com)

Optional but worth it for a paid product. The domain is the only cost; hosting and HTTPS stay free on Netlify.

1. Buy `jobrajobs.com` at a registrar. **Decline every add-on** (hosting, SSL, email, privacy upsells). You only want the bare domain, because Netlify gives you hosting and a free SSL certificate. GoDaddy’s first-year deal is fine; just know the renewal is roughly \$15–22/year (Cloudflare Registrar or Porkbun are cheaper long-term).
2. In Netlify: **Site → Domain management → Add a domain** and enter `jobrajobs.com`.
3. Point DNS. Easiest is to switch your registrar’s **nameservers** to the ones Netlify shows (this hands DNS to Netlify). Otherwise add the A/CNAME records Netlify lists at your registrar.
4. Netlify auto-issues a free HTTPS certificate once DNS resolves (a few minutes to a few hours). No action needed.

5. Update two URLs to the new domain:

- Supabase → Authentication → URL Configuration: set **Site URL** and add `https://jobrajobs.com` (and `https://www.jobrajobs.com`) to **Redirect URLs**, so magic-link sign-ins return to your domain.
- Your Stripe payment links' success URL (step in the payments section):
`https://jobrajobs.com/?subscribed=1`.

3. Fresh jobs (Adzuna + GitHub Actions)

1. Get a free API key at **developer.adzuna.com** (gives you an `app_id` and `app_key`; covers both US and Canada).
2. Push this folder to a **GitHub repo** (public = unlimited free Actions minutes).
3. In the repo: **Settings → Secrets and variables → Actions → New repository secret**, add four secrets:
 - `SUPABASE_URL`
 - `SUPABASE_SERVICE_ROLE_KEY` ← from Supabase **Settings → API → service_role** (keep secret, never put in the front-end)
 - `ADZUNA_APP_ID`
 - `ADZUNA_APP_KEY`
4. The workflow runs hourly automatically. To fill the feed right now, open the **Actions** tab → *Jobra job ingestion* → **Run workflow**.

To run ingestion on your own machine instead:

```
cd ingest && npm install
SUPABASE_URL=... SUPABASE_SERVICE_ROLE_KEY=... ADZUNA_APP_ID=... ADZUNA_APP_KEY=.
```

Adding a member

Until payments are wired up, add people manually: Supabase **Table Editor → subscribers → Insert row** with their email and `status = active`. To revoke, set `status` to `inactive`.

What it costs

Piece	Service	Cost
-------	---------	------

Hosting	Netlify free	\$0
Login + database	Supabase free	\$0 (500 MB DB, 5 GB egress)
Job data	Adzuna / USAJobs / Job Bank	\$0
Hourly refresh	GitHub Actions (public repo)	\$0
Payments (later)	Stripe	\$0/month , just 2.9% + 30¢ <i>per payment</i>

Watch-outs as you grow: Supabase's free limits are 500 MB of data and 5 GB of monthly egress. The ingestion script auto-prunes jobs older than ~3 weeks to stay well under that. If you outgrow it, Supabase Pro is ~\$25/mo. Everything else stays free far longer.

Payments & subscriptions (Stripe)

Pricing is three auto-renewing tiers, with the 3-month plan highlighted as “Most popular”:

Plan	Price	Renews	Per month
Monthly	\$50	every month	\$50
3 months	\$105	every 3 months	~\$35
Yearly	\$240	every year	~\$20

Everything gates on one field: `subscribers.status = 'active'` . Stripe just flips that field through a webhook, so the app needs no further changes.

One-time Stripe setup

1. In Stripe, create one product (“Jobra AI membership”) with three recurring **Prices**: \$50/month, \$105 every 3 months (set interval = month, interval count = 3), and \$240/year.
2. For each price, create a **Payment Link** (Stripe → Payment links). On each link, set the success URL to your site with `?subscribed=1` (e.g. `https://jobrajobs.com/?subscribed=1`) so the customer sees the “sign in now” banner when they return. Paste the three URLs into `JOBRA_CONFIG` (`STRIPE_LINK_MONTHLY` , `STRIPE_LINK_QUARTERLY` ,

`STRIPE_LINK_YEARLY`).

3. Enable the **Customer portal** (Stripe → Settings → Billing → Customer portal). Turn on “cancel subscription”. Copy its login link into `STRIPE_PORTAL_URL` , which powers the “Manage subscription” button.
4. Deploy the webhook function: `supabase functions deploy stripe-webhook --no-verify-jwt` . Set its secrets (`STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` , `SUPABASE_URL` , `SUPABASE_SERVICE_ROLE_KEY`).
5. In Stripe → Developers → Webhooks, add an endpoint pointing at the function URL, subscribing to `checkout.session.completed` , `customer.subscription.updated` , `customer.subscription.deleted` , and `invoice.payment_failed` .

After that it runs itself: someone pays → webhook sets them `active` → they sign in with the same email → feed unlocks. Renewals keep them active; a failed charge sets `past_due` ; a cancellation sets `inactive` .

How cancellation works

Members cancel themselves through the Stripe customer portal (the “Manage subscription” button). They can cancel immediately or at the end of the period they’ve paid for. You can also cancel anyone from the Stripe Dashboard (open their subscription → Cancel). Either way Stripe sends the webhook and the member’s access turns off automatically.

How pausing works

Stripe no longer lets customers pause themselves in the portal, so pausing is something you do: in the Stripe Dashboard, open the subscription and “pause payment collection”, then set that member’s `status` to `inactive` in Supabase while they’re paused. Resume both when they’re back. (A true self-serve pause would be a small custom build via the Stripe API.)

What Stripe tells you

You’re notified three ways at once: a real-time webhook to the app (the automatic part), an email on each successful payment, and a searchable, CSV-exportable customer and subscription list in the Stripe Dashboard.

Running it manually first (before deploying the webhook)

You don’t have to wire the webhook on day one. You can take payments with the Payment Links and just add members by hand: when Stripe emails you about a sale, open Supabase → Table Editor → subscribers → Insert row with their email and `status = active` . Deploy the webhook once that gets tedious.

Prefer Shopify? It can do the same via its order/subscription webhooks, but it carries a monthly plan fee plus subscription-app fees, so it's not the \$0 choice. Stripe charges \$0/month and only takes a cut per payment.

Résumé upload & match scoring (built in, \$0)

Members go to the **My résumé** tab and upload a PDF or `.docx` (or paste the text).

Everything below runs **in the browser**, with no AI API, no keys, and no per-use cost, and the résumé never leaves their device unless they save it:

- **Parsing:** PDF via pdf.js, Word via mammoth.js (both loaded free from a CDN).
- **Four match scores per job:** every role gets a **Résumé match %** (the headline ring), which breaks down into **Skill match**, **Experience level match**, and **Industry match** (tap the ring to expand). Skill match compares detected skills to each job's requirements; experience match compares seniority/years; industry match compares domains; the overall blends these with a text-similarity score.
- **Sort by Best match:** once a résumé is loaded, the feed re-ranks by fit.
- **Résumé checkup:** local résumé checks (length, contact info, quantified impact, action verbs, skill coverage) with concrete fixes.
- **Skill gaps:** the most requested skills across the member's matching roles that aren't on their résumé yet.

If logged in (live mode), the parsed résumé is saved to the `profiles` table so it persists; in preview mode it stays in memory.

Applied jobs tracker

A third tab, "Applied", is each member's application tracker. When they open a role from the feed (it launches the external posting in a new tab) and then come back to Jobra, a small prompt asks "Did you apply to {role}?" with two choices: "Yes, mark applied" or "Not yet" (which saves it to follow up on). Tracked jobs appear in the Applied tab, each with a status they can change at any time (saved → applied → interviewing → offer → rejected) and a button to remove it. The tab shows a live count. In live mode this is stored in the `applications` table per member; in preview it's in memory.

Signing in

Subscribing needs no account: the plans are on the public landing page and anyone can

pay. Signing in is only for opening the feed afterward. The “Sign in” button in the top bar (and the link under the plans) opens a sign-in modal, so it’s reachable from anywhere without scrolling. After paying, returning members are dropped straight onto it.

Optional: AI tailoring (still \$0)

The **AI tailoring** button rewrites the summary and bullets for a member’s top-matched roles.

- **In the Claude.ai preview**, it works out of the box (uses Claude’s API, no key).
- **On your live site**, point `AI_REFINE_ENDPOINT` (top of `index.html`) at a tiny serverless proxy (a free Supabase Edge Function) that calls a free-tier LLM:
 - **Google Gemini**: about 1,500 requests/day free, no credit card.
 - **Groq**: about 14,400 requests/day free, no credit card.
 - Keep the LLM key inside the Edge Function (never in the front-end). The function takes `{prompt}` and returns `{text}` .
 - Privacy note: free LLM tiers may use prompts for training, so tell members their résumé text is sent to that provider when they use AI tailoring (the local scoring/checkup never sends anything).

Roadmap (what’s next)

- **Phase 1**: add USAJobs + Job Bank Canada sources to the pipeline.
- **Phase 2**: crawl company ATS feeds (Greenhouse / Lever / Ashby) for hourly, high-quality roles.
- **Phase 3**: email/alert digests of new high-match roles, and Stripe self-serve checkout.